

Mid-Course Assignment:

Custom RAG Pipeline on Your Own Data

Goal

In this assignment, you will implement a complete **Retrieval-Augmented Generation (RAG)** pipeline over a dataset of your choice.

The goal is to understand and control the full RAG pipeline:

1. data collection
2. document loading
3. text cleaning and preprocessing
4. chunking
5. embedding
6. indexing
7. retrieval
8. answer generation
9. citation of sources
10. evaluation and error analysis

Your system should answer questions using your own corpus and provide evidence from the retrieved documents.

Dataset / Corpus

You must choose a corpus that is personally or professionally meaningful to you.

Examples:

- technical documentation of a tool or library
- course notes or lecture material
- research papers on a specific topic
- internal documents, after removing private information
- legal, historical, medical, financial, or domain-specific documents
- documentation of an open-source project
- personal knowledge base, if privacy is handled carefully

The corpus should contain enough information to make retrieval meaningful.

Minimum requirement:

At least 30 pages of text

or

at least 50 meaningful documents / sections / records

The corpus should not be a generic Wikipedia-style dataset that a strong LLM already knows well. The attached proposal emphasizes that the corpus should contain knowledge where a baseline LLM may be wrong, vague, or incomplete without retrieval.

Required System Interface

Your RAG system must expose the following function:

```
def answer(question: str) -> dict:
    return {
        "answer": str,
        "sources": list[str],
        "retrieved_chunks": list[dict]
    }
```

Where:

- `answer`: the final natural-language answer
- `sources`: identifiers of the documents/chunks used to answer
- `retrieved_chunks`: the retrieved chunks with metadata

Example:

```
{
  "answer": "The model uses residual connections to improve gradient flow...",
  "sources": ["doc_03_chunk_12", "doc_07_chunk_04"],
  "retrieved_chunks": [
    {
      "chunk_id": "doc_03_chunk_12",
      "text": "...",
      "score": 0.82,
      "metadata": {
        "source": "resnet_paper.pdf",
        "page": 4
      }
    }
  ]
}
```

This interface follows the idea in the proposal of a fixed `eval_runner.py` that can evaluate all student systems through the same `answer()` function.

Required Files

Your submission should include the following structure:

```
project/
├── data/
│   ├── raw/
│   ├── processed/
│   └── MANIFEST.md
├── src/
│   ├── build_index.py
│   ├── rag_system.py
│   ├── retrieval.py
│   ├── generation.py
│   └── utils.py
├── eval/
│   ├── gold_set.jsonl
│   └── run_eval.py
├── report.pdf
├── requirements.txt
└── README.md
```

Part 1 Requirements

1. Data Preparation

Create a `MANIFEST.md` file describing your corpus.

It should include:

Corpus name:
Domain:
Source of documents:
Number of documents:
Approximate number of pages / tokens:
File types:
License / permission:
Why this corpus is suitable for RAG:
What kind of questions should the system answer:

If the corpus contains private or sensitive information, you must remove it before submission.

2. Document Loading

Implement a loading pipeline that reads your documents and converts them into a clean internal format.

Each loaded document should contain:

```
{
  "doc_id": str,
  "text": str,
  "metadata": dict
}
```

Metadata may include:

```
{
  "source": "filename.pdf",
  "page": 3,
  "section": "Introduction",
  "date": "2024-05-01"
}
```

3. Chunking

Split the documents into chunks.

You must experiment with at least two chunking strategies.

For each chunk, store:

```
{
  "chunk_id": str,
  "doc_id": str,
  "text": str,
  "metadata": dict
}
```

In your report, explain:

- chunk size
- overlap size if any
- why you chose this strategy
- one example where chunking helped or hurt retrieval

4. Embedding and Indexing

Create embeddings for your chunks and store them in a vector index.

You may use any vector database or local index, like FAISS, Chroma, Qdrant etc. or simple NumPy-based search, for small corpora

Your `build_index.py` script should build the index from scratch:

The script should be reproducible. If we delete the index and run the script again, it should rebuild the same index from the data.

5. Retrieval

Implement a retriever that receives a question and returns the top-k most relevant chunks.

Minimum requirement:

```
retrieve(query: str, k: int = 5) -> list[dict]
```

Each retrieved result should include:

```
{
  "chunk_id": str,
  "text": str,
  "score": float,
}
```

```
    "metadata": dict
}
```

You should report at least one retrieval metric.

6. Answer Generation

Use an LLM to generate an answer from the retrieved chunks.

The answer should follow these rules:

1. Answer only from the retrieved context.
2. If the answer is not in the context, say that the information was not found.
3. Cite the source chunks used in the answer.
4. Avoid unsupported claims.

Example prompt structure:

You are a question-answering assistant.
Answer the question using only the provided context.
If the answer is not found in the context, say so.

Question:
{question}

Context:
{retrieved_chunks}

Answer:

7. Gold Set

Create a small evaluation set of at least 50 questions.

Each line in `gold_set.jsonl` should contain:

```
{
  "question": "What problem does residual learning solve?",
  "reference_answer": "Residual learning addresses the degradation problem in deep neural networks.",
  "must_cite_chunk_ids": ["doc_01_chunk_07"],
  "category": "factual"
}
```

Use several question types:

- factual
- numerical
- temporal
- negation / absence
- comparison

8. Evaluation

Your evaluation should include:

Retrieval evaluation

Measure whether the system retrieves the correct chunks.

Answer evaluation

You should manually inspect at least 10 answers and classify them as:

Correct
Partially correct
Incorrect
Unsupported / hallucinated

Optional: use an LLM-as-judge, but manual analysis is enough for Part 1.

9. Ablation Study

Run at least two small ablation experiments.

Examples:

- chunk size 300 vs 700
- overlap 0 vs overlap 100
- embedding model A vs embedding model B
- top-k = 3 vs top-k = 8
- with vs without reranking
- dense retrieval vs BM25 vs hybrid retrieval

Report the result in a table:

Experiment	Hit@5	Answer accuracy	Notes
chunk size 300	0.72	0.60	better for short factual questions
chunk size 700	0.80	0.66	better context, more noise
top-k = 3	0.68	0.62	sometimes misses evidence
top-k = 8	0.84	0.64	retrieves more but adds noise

Final Report

Submit a short report, up to 4 pages.

The report should include:

1. corpus description
2. system architecture
3. chunking strategy
4. embedding and vector index choice
5. retrieval method

6. prompt design
7. evaluation results
8. ablation table
9. failure analysis
10. what you would improve next

Grading Rubric

Component	Points
Corpus selection and MANIFEST .md	10
Data loading and preprocessing	10
Chunking implementation and explanation	10
Embedding and index construction	15
Retrieval implementation	15
Answer generation with citations	15
Gold set and evaluation	15
Ablation study and failure analysis	10
Total	100

Submission

Submit a Git repository or zip file containing:

```
code
data manifest
gold_set.jsonl
README.md
requirements.txt
final report
```

The README must include exact running instructions:

```
pip install -r requirements.txt
python src/build_index.py
python eval/run_eval.py
```

Important Notes

- You may use LangChain, LlamaIndex, Haystack, or implement the pipeline manually.
- You must understand and explain every major component you use.
- Do not submit a black-box wrapper around a framework.
- The system must provide citations to retrieved chunks.
- If the answer is not in the corpus, the system should say so.
- Private data must be removed before submission.